

Київський національний університет
імені Тараса Шевченка

Факультет комп'ютерних наук та кібернетики

Звіт

до лабораторної роботи №1
на тему:

**«Розробка клієнта для взаємодії з віддаленою папкою з файлами. Етап 1,
варіант 8-6»**

Студента четвертого курсу
групи МІ-41
Мекшакова Артемія Олексійовича

Київ-2026

Зміст

1	Постановка задачі	1
2	Аналіз вимог	2
2.1	Дійові особи	2
2.2	Функціональні вимоги	2
2.3	Нефункціональні вимоги	2
2.4	Перелік прецедентів	3
3	Високорівневе проєктування	3
3.1	Архітектура	3
3.2	Рис. 1. Діаграма компонентів	4
3.3	Рис. 2. Діаграма розгортання	5
4	Деталізоване проєктування	6
4.1	Рис. 3. Діаграма прецедентів	6
4.2	Рис. 4. Діаграма класів: доменна модель	7
4.3	Рис. 5. Діаграма класів VOPC: сортування і фільтр	8
4.4	Рис. 6. Діаграма активності: синхронізація	9
4.5	Рис. 7. Діаграма послідовності: синхронізація	10
4.6	Рис. 8. Діаграма послідовності: перегляд вмісту файлу	11
4.7	Рис. 9. Діаграма комунікації: сортування і фільтр	12
4.8	Рис. 10. Діаграма станів файлу	13
5	Висновки	14

1 Постановка задачі

Робота виконується в межах курсу «Інформаційні технології» і полягає у створенні системи типу **2** — клієнта для взаємодії з віддаленою папкою з файлами, тобто полегшеного аналога Google Drive. Робоча назва проєкту — **Shelf**. Система складається з віддаленого сервера, який зберігає файли та їхні метадані, і двох клієнтів — десктопного і веб-клієнта, — що працюють з тим самим сервером через спільний REST API.

Індивідуальну частину визначає варіант **8-6** — дві останні цифри номера студентського квитка:

Цифра	Список	Значення
8	ТИПИ файлів, вміст яких показується при кліку	. kt — як текст, . jpg — як зображення
6	ОПЕРАЦІЇ	сортування за назвою (зростання / спадання); фільтр «усі файли» / «лише . crr» / «лише . png»

Роботу поділено на чотири етапи — UML-специфікація (2 бали), десктопна версія з unit-тестами (13), веб-версія (13) і порівняльний аналіз (2); обидва бонуси, drag-and-drop під

час завантаження і скачування та публікація сервера за публічною адресою (по +5 балів), заплановано й враховано в специфікації.

Нюанс варіанта. Типи файлів для перегляду (.kt, .jpg) і для фільтра (.cpp, .png) не збігаються, бо цифри варіанта незалежні. Тому перегляд реалізується узагальнено: будь-який текстовий файл показується як текст, будь-яке растрове зображення — як картинка, а для решти типів відкриваються лише атрибути з поміткою «перегляд недоступний». Обов'язковими й покритими тестами випадками залишаються саме .kt та .jpg.

Обов'язковий мінімум діаграм UML-специфікації етапу 1:

- прецедентів — 1+ (у звіті Рис. 3);
- класів — 2+, зокрема VOPC (Рис. 4, 5);
- активності — 1+ (Рис. 6);
- взаємодії — 2+ (Рис. 7, 8, 9);
- станів — 1+ (Рис. 10);
- компонентів — 1+ (Рис. 1);
- розгортання — 1+ (Рис. 2).

2 Аналіз вимог

2.1 Дійові особи

Єдина дійова особа — **User**, користувач з обліковим записом на сервері; без входу йому доступні лише реєстрація та вхід. Адміністратора немає, бо кожен бачить тільки власний простір, а база даних і об'єктне сховище — внутрішні компоненти, а не актори.

2.2 Функціональні вимоги

ID	Вимога	Прецеденти
R1	Реєстрація, вхід і вихід; власний простір (віртуальний диск)	UC1–UC3, UC15
R2	Список файлів з атрибутами: назва, дата створення, дата зміни, хто завантажив, хто редагував	UC4
R3	Показ і приховування будь-якого стовпця, крім назви	UC7
R4	Сортування за назвою за зростанням і спаданням (варіант)	UC5
R5	Фільтр: усі файли, лише .cpp, лише .png (варіант)	UC6
R6	Перегляд вмісту: .kt як текст, .jpg як зображення (варіант)	UC8
R7	Завантаження файлу або нової версії, скачування, видалення	UC9–UC12
R8	Синхронізація локальної папки з розв'язанням конфліктів	UC13, UC14
R9	Бонус: drag-and-drop під час завантаження і скачування	UC9b, UC11a

2.3 Нефункціональні вимоги

- **Безпека:** bcrypt-хеші паролів, сесія JWT, перевірка власника в кожному запиті.
- **Обмеження:** файл до 50 МБ; плоский простір без підпапок; те саме ім'я — нова версія.

- **Переносимість:** Electron на macOS і Windows; синхронізація у веб — Chrome та Edge.
- **Тестованість:** логіка списку й синхронізації — чисті функції, тестовані без сервера.

2.4 Перелік прецедентів

Прецедент	Опис
UC1. Sign up	реєстрація і створення простору
UC2. Log in to the system	вхід за поштою і паролем, отримання JWT
UC3. Work with the file storage	узагальнення: вхід і перегляд списку
UC4. View the list of files and their attributes	таблиця файлів з усіма атрибутами
UC5. Sort by name	за зростанням або спаданням (варіант)
UC6. Filter by type	усі файли, лише .cpp, лише .png
UC7. Show / hide table columns	видимість стовпців, крім назви
UC8. View the file contents	.kt як текст, .jpg як зображення
UC9. Upload file(s) to the storage	UC9a — діалог, UC9b — drag-and-drop
UC10. Update a file (new version)	те саме ім'я оновлює дату зміни й редактора
UC11. Download file(s)	скачування; UC11a — перетягування з вікна
UC12. Delete file(s)	видалення запису й об'єкта у сховищі
UC13. Bind / change local folder	вибір або зміна папки синхронізації
UC14. Synchronize with the local folder	UC14a — відстеження, UC14b — конфлікти
UC15. Log out from the system	вихід і знищення сесії на клієнті

Прецеденти UC5 і UC6 деталізовано на Рис. 5 і 9, UC8 — на Рис. 8, UC14 — на Рис. 6 і 7.

3 Високорівневе проєктування

3.1 Архітектура

Система реалізується як монорепозіторій на `pnpm workspaces`. Обидва клієнти працюють з одним REST API (`/api`) без власної бізнес-логіки: сортування, фільтр, видимість стовпців, вибір перегляду і план синхронізації обчислюються **на клієнті**, у пакеті `@shelf/shared`, тож операції варіанта тестуються без мережі, а обидві версії поведуться однаково. Сервер на NestJS відповідає за автентифікацію, метадані, байти й контрольні суми та складається з модулів `AuthModule`, `UsersModule`, `WorkspaceModule`, `FilesModule`, `StorageModule` і `PrismaModule`.

`shelf/`

<code>apps/api/</code>	NestJS 11, Prisma, PostgreSQL, S3, JWT
<code>apps/desktop/</code>	Electron + React + Vite
<code>apps/web/</code>	Next.js (App Router)
<code>packages/shared/</code>	@shelf/shared — типи і чисті функції
<code>packages/ui/</code>	@shelf/ui — спільні React-компоненти
<code>docker/</code>	compose: postgres, minio, api

3.2 Рис. 1. Діаграма компонентів

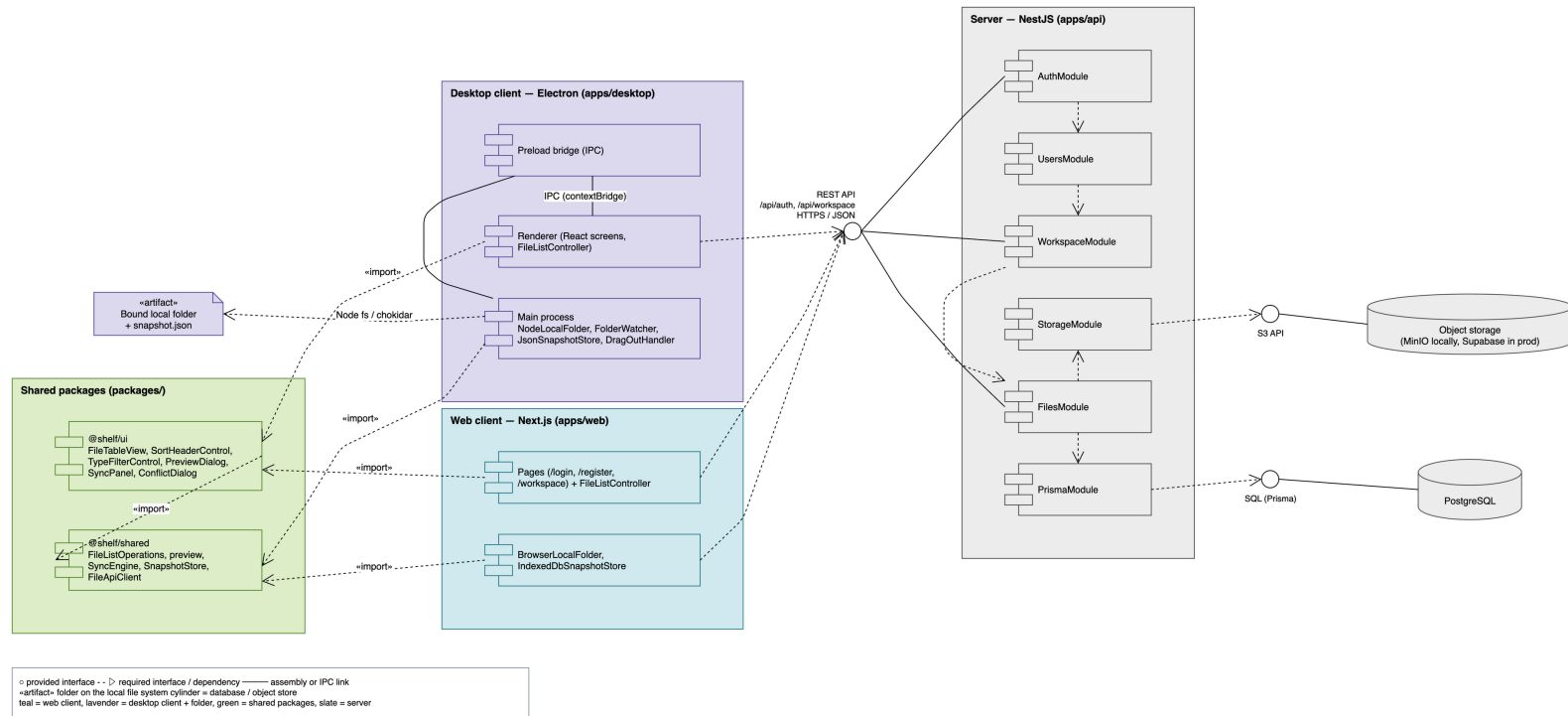


Рисунок 1. Діаграма компонентів

Пояснення. Діаграма поділяє систему на чотири підсистеми. Спільні пакети містять @shelf/ui з React-компонентами «FileTableView», «PreviewDialog», «SyncPanel», «ConflictDialog» та іншими і @shelf/shared з «FileListOperations», «SyncEngine», інтерфейсом «SnapshotStore» і «FileApiClient», тож логіка списку й синхронізації зосереджена в одному місці. Desktop-клієнт складається з main-процесу («NodeLocalFolder», «FolderWatcher», «JsonSnapshotStore», «DragOutHandler»), моста «Preload bridge (IPC)» і рендера з «FileListController»; веб-клієнт — зі сторінок з тим самим контролером і компонента «BrowserLocalFolder, IndexedDbSnapshotStore». Обидва клієнти споживають єдиний інтерфейс «REST API». Сервер містить шість модулів NestJS, з яких «StorageModule» вимагає «S3 API» до об'єктного сховища, а «PrismaModule» — «SQL (Prisma)» до PostgreSQL.

3.3 Рис. 2. Діаграма розгортання

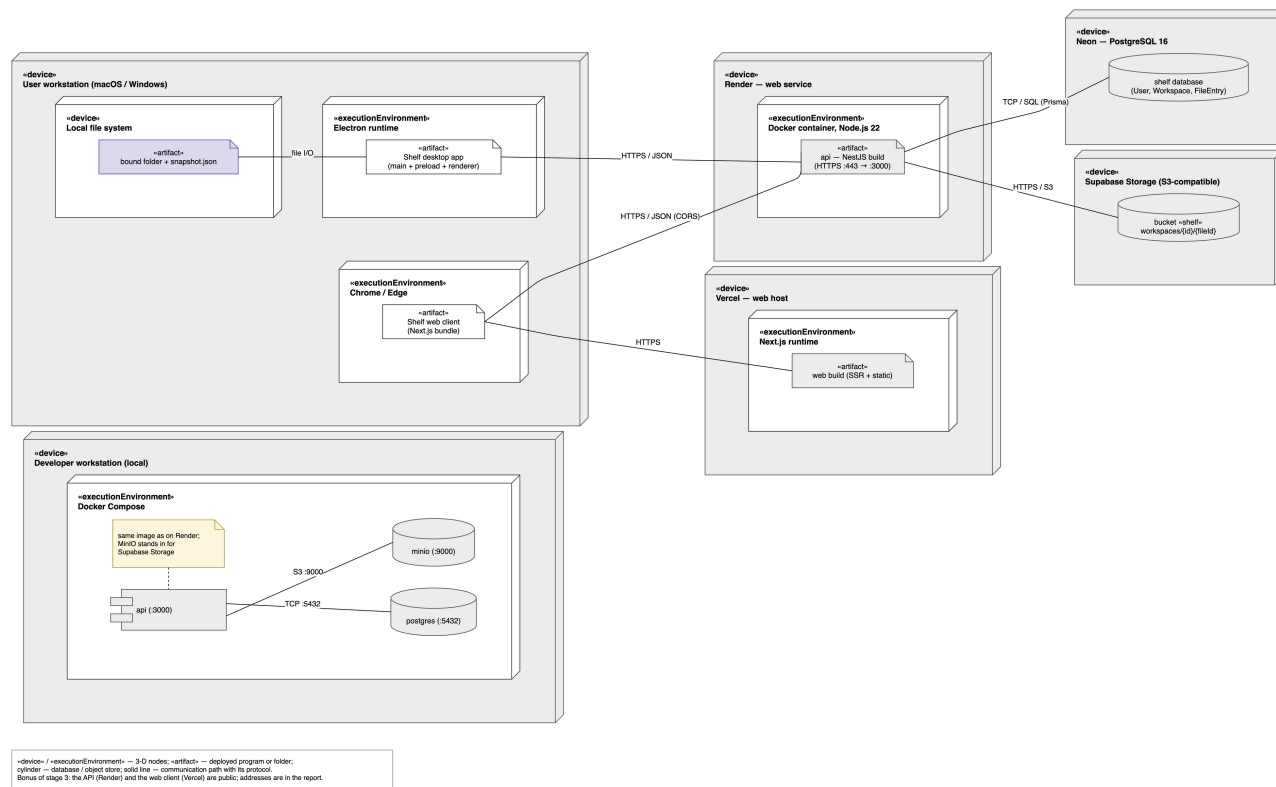


Рисунок 2. Діаграма розгортання

Пояснення. Вузли позначено стереотипами «device» і «executionEnvironment», а розгорнуті програми — «artifact». Робоча станція користувача (macOS або Windows) містить «Electron runtime» з десктопним застосунком, браузер «Chrome / Edge» з веб-клієнтом і локальну файлову систему з артефактом «bound folder + snapshot.json», прямий доступ «file I/O» до якої має лише десктоп. Обидва клієнти звертаються по «HTTPS / JSON» (браузер — з CORS) до вузла «Render — web service», де в контейнері Docker з Node.js 22 працює збірка NestJS. Від нього йдуть зв'язки «TCP / SQL (Prisma)» до «Neon — PostgreSQL 16» і «HTTPS / S3» до «Supabase Storage». Сторінки веб-клієнта віддає «Vercel — web host». На станції розробника «Docker Compose» піднімає той самий образ api з postgres і MinIO замість Supabase Storage.

4 Деталізоване проєктування

4.1 Рис. 3. Діаграма прецедентів

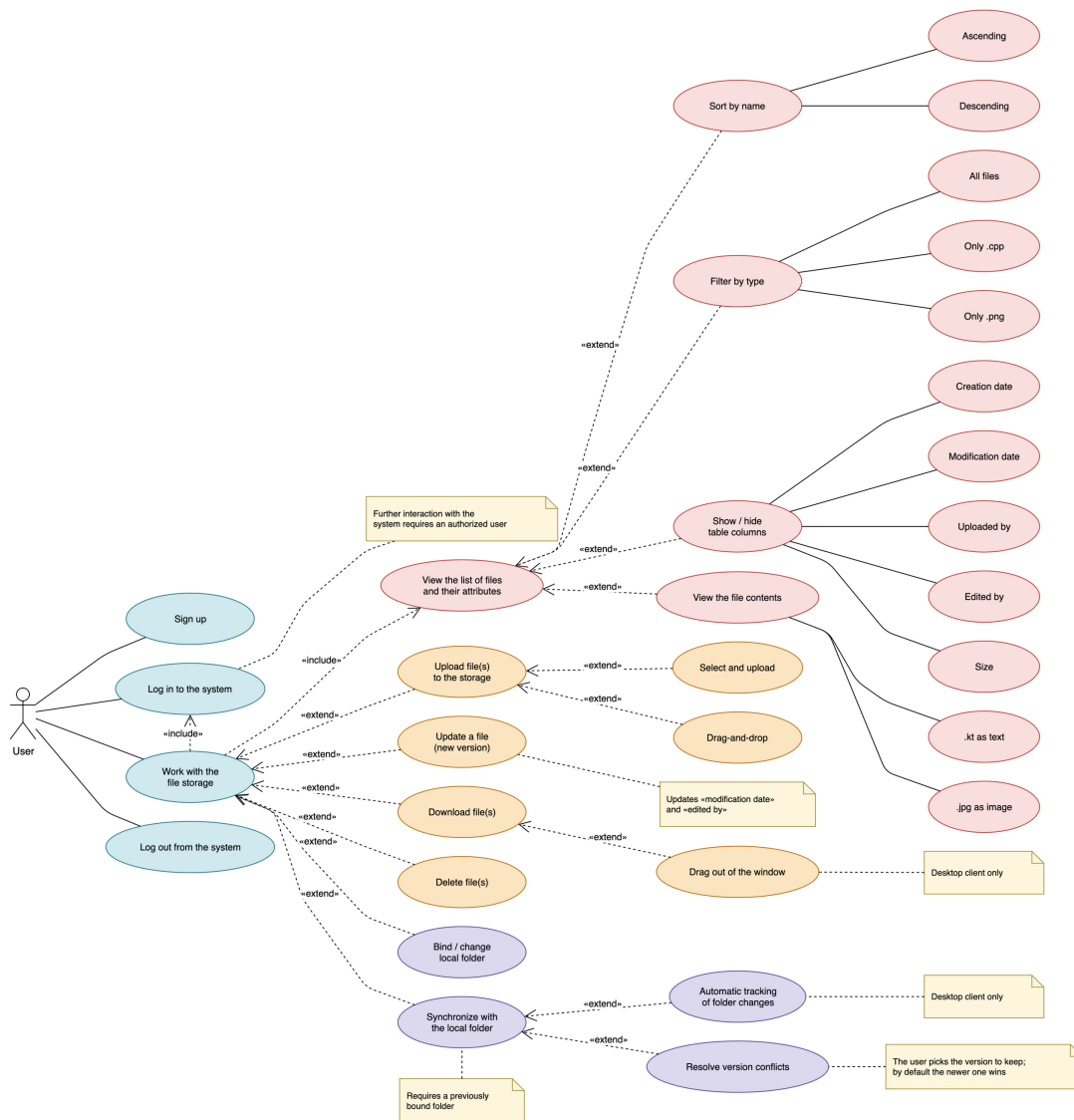


Рисунок 3. Діаграма прецедентів

Пояснення. Прецеденти впорядковані каскадом: ліворуч «User», у центрі базові прецеденти, праворуч їхні уточнення. Група access містить «Sign up», «Log in to the system», «Work with the file storage» і «Log out from the system»; третій з них через «include» охоплює вхід і «View the list of files and their attributes». Від перегляду списку відходять розширення групи list: операції варіанта «Sort by name» і «Filter by type» з листками «Only .cpp» та «Only .png», «Show / hide table columns» і «View the file contents» з листками «.kt as text» та «.jpg as image». Група files охоплює завантаження з листком «Drag-and-drop», нову версію, скачування і видалення, а група sync — прив'язку папки та синхронізацію з розширенням «Resolve version conflicts».

4.2 Рис. 4. Діаграма класів: доменна модель

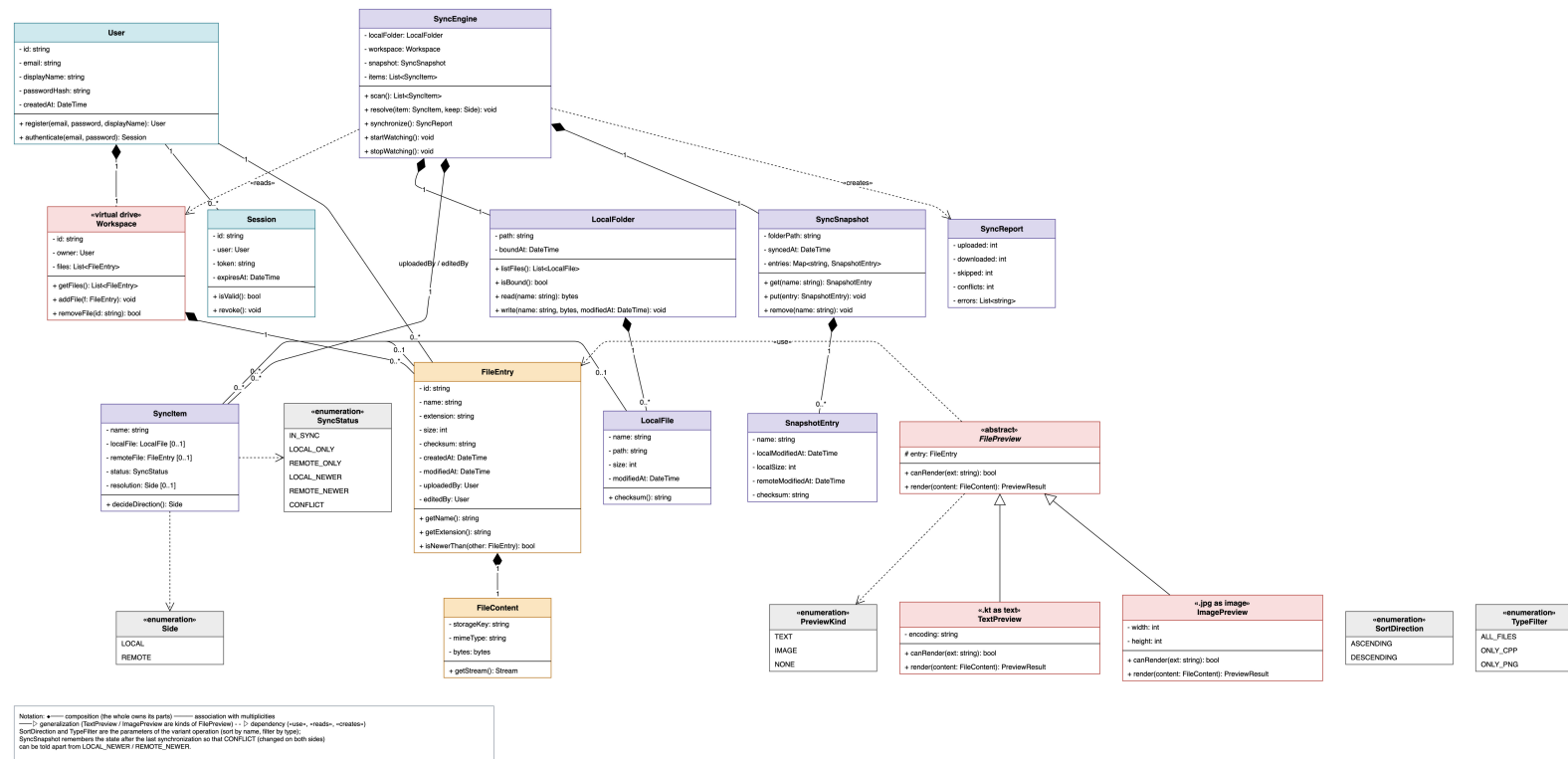


Рисунок 4. Діаграма класів: доменна модель

Пояснення. Модель має три групи класів. Облік і зберігання: «User» з `register` і `authenticate`, «Session» з токеном, «Workspace» зі стереотипом «virtual drive» (простір користувача), «FileEntry» — метадані файлу з `checksum`, датами і посиланнями `uploadedBy` та `editedBy`, і «FileContent» — байти зі `storageKey` і `mediaType`; композиція «Workspace» → «FileEntry» → «FileContent» означає, що частина не існує без цілого. Перегляд: абстрактний «FilePreview» з `canRender(ext)` і `render(content)` та спадкоємці «TextPreview» і «ImagePreview». Синхронізація: «LocalFolder» і «LocalFile», «SyncEngine» зі списком «SyncItem» і звітом «SyncReport», а також «SyncSnapshot» із записами «SnapshotEntry» — знімок стану після останньої синхронізації, який відрізняє `CONFLICT` від `LOCAL_NEWER` чи `REMOTE_NEWER`. Переліки винесено в окремі блоки «enumeration».

4.3 Рис. 5. Діаграма класів VOPC: сортування і фільтр

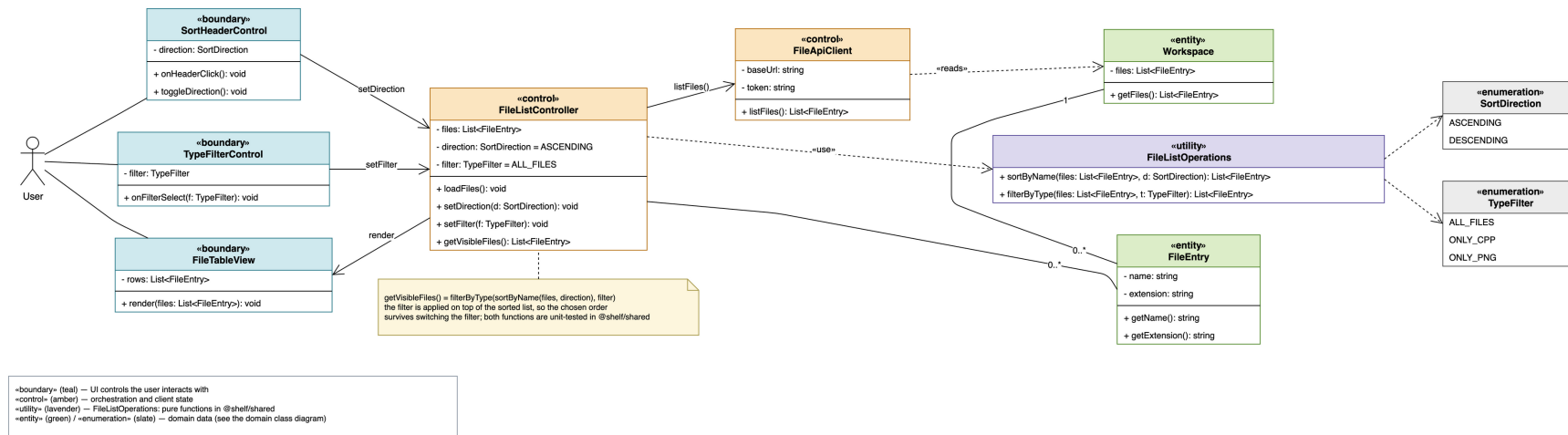


Рисунок 5. Діаграма класів VOPC: сортування і фільтр

8

Пояснення. Діаграма містить лише класи прецедентів «Sort by name» і «Filter by type», розділені за стереотипами. Межові класи «boundary» — «SortHeaderControl» з `toggleDirection()`, «TypeFilterControl» з `onFilterSelect(f)` і «FileTableView» з `render(files)` — безпосередньо взаємодіють з «User». Керувальний «FileListController» тримає `files`, `direction = ASCENDING` і `filter = ALL_FILES` та надає `loadFiles()`, `setDirection(d)`, `setFilter(f)` і `getVisibleFiles()`, а «FileApiClient» читає список викликом `listFiles()`. Службовий «FileListOperations» містить чисті функції `sortByName` і `filterByType`, на які пишуться unit-тести етапу 2; сутності «Workspace», «FileEntry» і переліки взято з доменної моделі. Примітка фіксує інваріант `getVisibleFiles() = filterByType(sortByName(files, direction), filter)`: фільтр застосовується до вже відсортованого списку, тож порядок не губиться.

4.4 Рис. 6. Діаграма активності: синхронізація

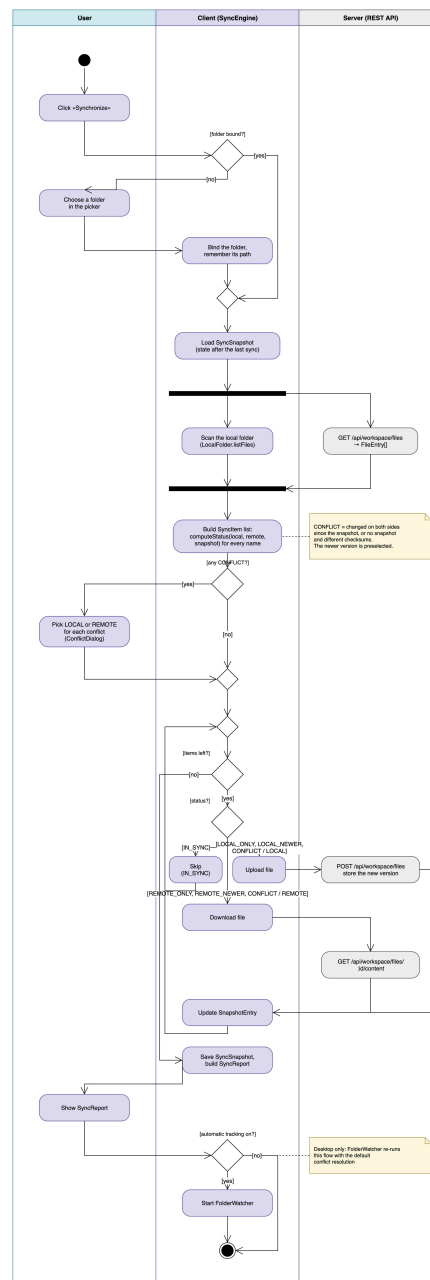


Рисунок 6. Діаграма активності: синхронізація

Пояснення. Діаграма має три доріжки: «User», «Client (SyncEngine)» і «Server (REST API)». Після натискання «Synchronize» вузол рішення [folder bound?] за потреби пропонує обрати й прив'язати папку, далі завантажується «SyncSnapshot». Вузол розділення (fork) паралельно запускає сканування локальної папки і GET /api/workspace/files, вузол з'єднання (join) чекає обидві дії, після чого computeStatus(local, remote, snapshot) буде список «SyncItem». Конфлікти користувач розв'язує в «ConflictDialog». Цикл обходить план: IN_SYNC пропускається, LOCAL_ONLY, LOCAL_NEWER і CONFLICT / LOCAL вивантажуються, REMOTE_ONLY, REMOTE_NEWER і CONFLICT / REMOTE скачуються, а після кожного перенесення оновлюється «SnapshotEntry». Наприкінці знімок зберігається, показується «SyncReport», а за ввімкненого автовідстеження запускається «FolderWatcher».

4.5 Рис. 7. Діаграма послідовності: синхронізація

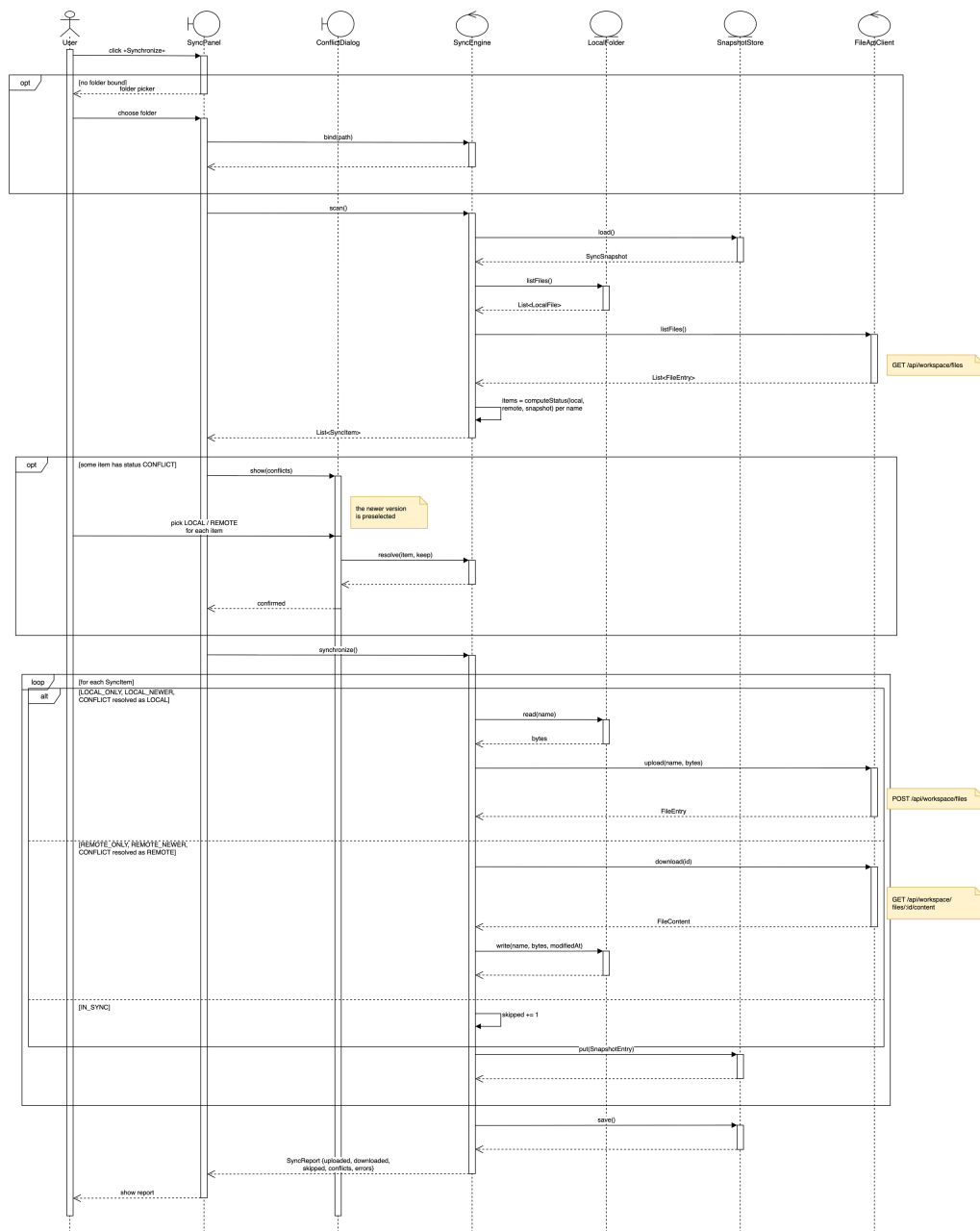


Рисунок 7. Діаграма послідовності: синхронізація

Пояснення. Той самий сценарій показано обміном повідомленнями між сімома лініями життя: «User», «SyncPanel», «ConflictDialog», «SyncEngine», «LocalFolder», «SnapshotStore» і «FileApiClient». Фрагмент opt [no folder bound] описує вибір папки і bind(path). Виклик scan() читає знімок «SyncSnapshot» повідомленням load() до «SnapshotStore», отримує списки від LocalFolder.listFiles() і FileApiClient.listFiles() та самовикликом computeStatus повертає List<SyncItem>. Фрагмент opt [some item has status CONFLICT] показує show(conflicts) і resolve(item, keep). Далі synchronize() розгортається в loop [for each SyncItem] з alt на три гілки: upload через POST, download і write через GET, skipped += 1 для IN_SYNC. Після кожного перенесення знімок оновлюється викликом put(SnapshotEntry), після циклу — save(), а «SyncPanel» показує «SyncReport».

4.6 Рис. 8. Діаграма послідовності: перегляд вмісту файлу

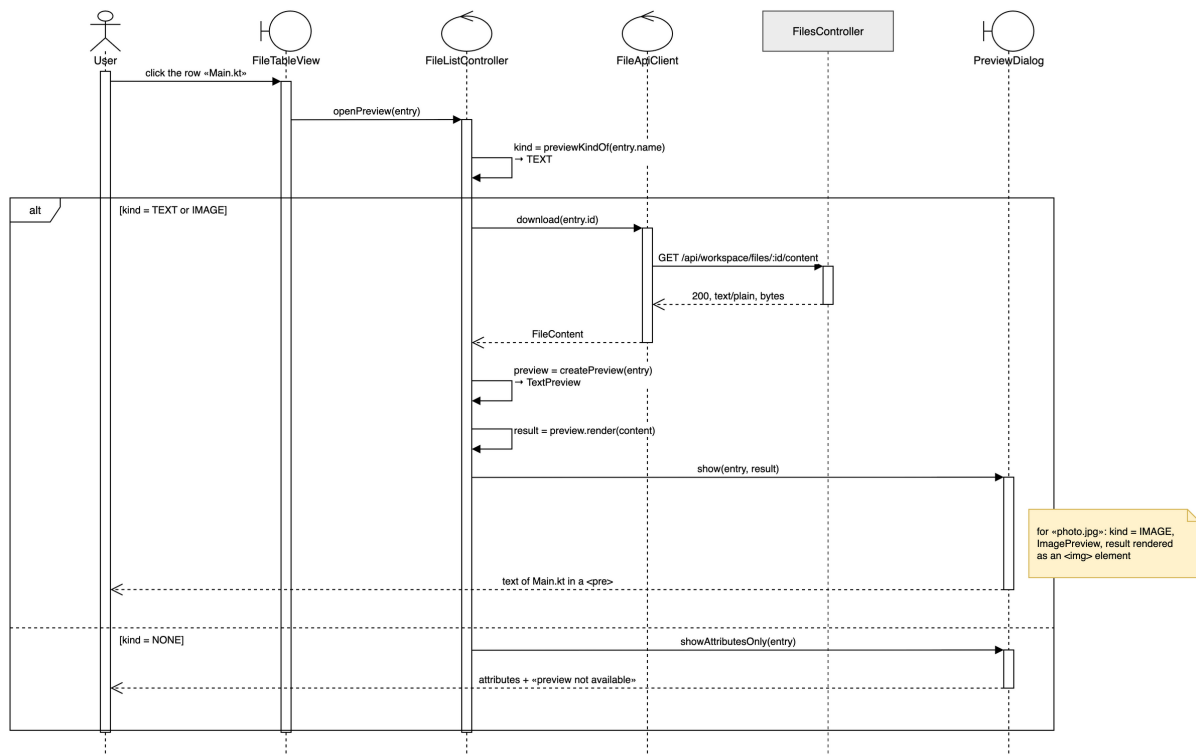


Рисунок 8. Діаграма послідовності: перегляд вмісту файлу

Пояснення. Сценарій починається кліком «User» по рядку «Main.kt» у «FileTableView», яка викликає `openPreview(entry)` у «FileListController». Самовиклик `previewKindOf(entry.name)` повертає `TEXT`: ця чиста функція визначає спосіб перегляду лише за розширенням, не звертаючись до сервера. У гілці `[kind = TEXT or IMAGE]` фрагмента `alt` «FileApiClient» виконує `GET /api/workspace/files/:id/content` до «FilesController», отримує відповідь «200, text/plain, bytes» і повертає «FileContent»; `createPreview(entry)` створює «TextPreview», а `render(content)` дає результат, який «PreviewDialog» показує як текст у `<pre>`. Примітка описує симетричний випадок `.jpg`: для «photo.jpg» створюється «ImagePreview», а результат виводиться як `<img>`. Гілка `[kind = NONE]` викликає `showAttributesOnly(entry)` — лише атрибути з позначкою «preview not available».

4.7 Рис. 9. Діаграма комунікації: сортування і фільтр

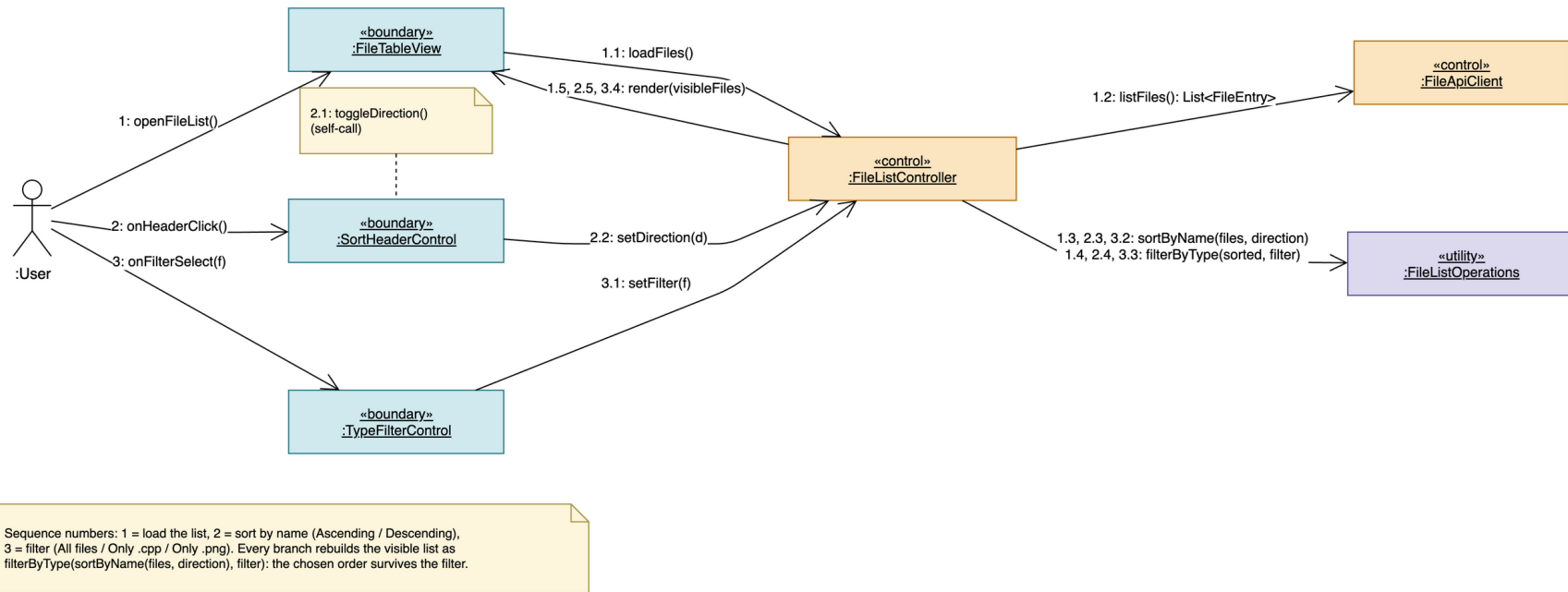


Рисунок 9. Діаграма комунікації: сортування і фільтр

Пояснення. Діаграма показує ті самі об'єкти, що й діаграма VOPC, зі стереотипами «boundary», «control» і «utility», але наголошує на зв'язках і нумерації повідомлень. Перша послідовність — завантаження списку: 1: openFileList() до :FileTableView, 1.1: loadFiles() до :FileListController, 1.2: listFiles() до :FileApiClient, потім :FileListOperations отримує 1.3: sortByName і 1.4: filterByType, а :FileTableView — 1.5: render(visibleFiles). Друга починається з 2: onHeaderClick(), самовиклику 2.1: toggleDirection() і 2.2: setDirection(d), третя — з 3: onFilterSelect(f) і 3.1: setFilter(f). Завершальний ланцюжок не дублюється, а перевикористовується, тому зв'язки мають потрібну нумерацію «1.3, 2.3, 3.2», «1.4, 2.4, 3.3» і «1.5, 2.5, 3.4». Примітка внизу розшифровує номери: усі три дії перебудовують список тим самим виразом, тож обраний порядок переживає зміну фільтра.

4.8 Рис. 10. Діаграма станів файлу

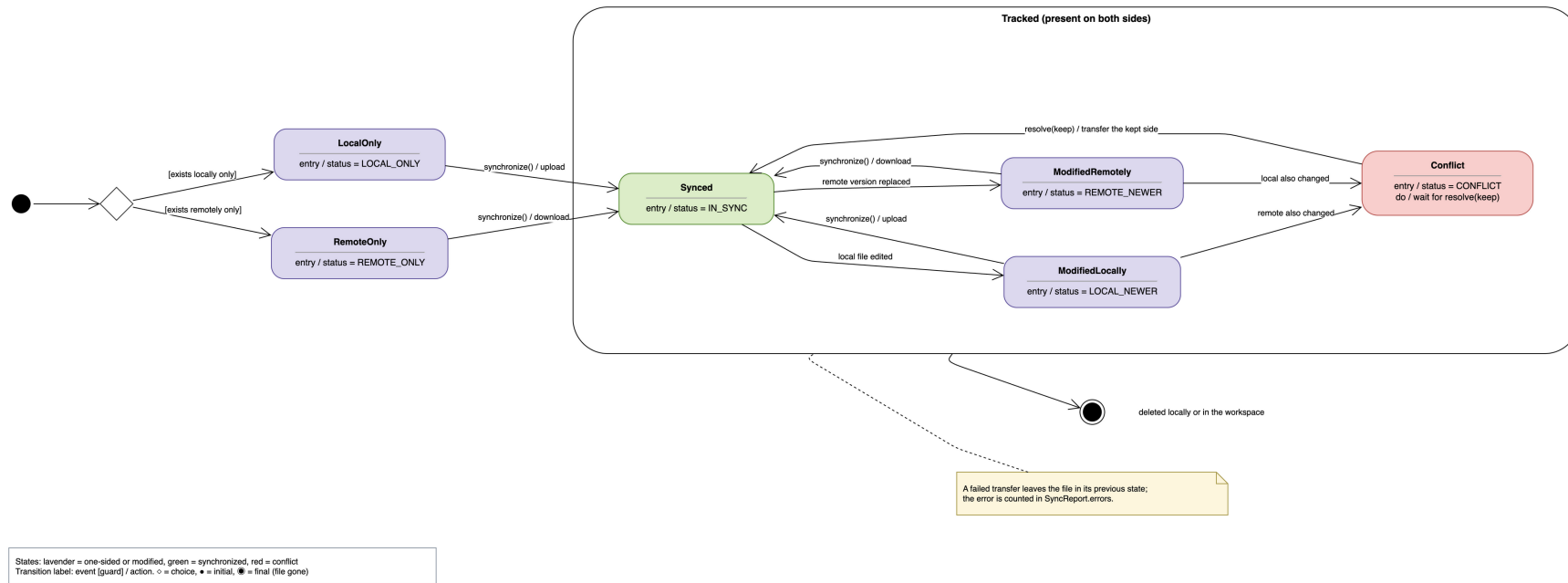


Рисунок 10. Діаграма станів файлу

Пояснення. Діаграма описує життєвий цикл одного імені файлу під час синхронізації. З початкового псевдостану вузол вибору за умовою [exists locally only] веде у стан «LocalOnly» (LOCAL_ONLY), а за [exists remotely only] — у «RemoteOnly» (REMOTE_ONLY). Переходи synchronize() / upload і synchronize() / download переводять файл у композитний стан «Tracked» з підстанами «Synced» (IN_SYNC), «ModifiedLocally» (LOCAL_NEWER), «ModifiedRemotely» (REMOTE_NEWER) і «Conflict» (CONFLICT). Зміна з одного боку виводить файл зі «Synced», а наступна синхронізація повертає його назад. У «Conflict» він потрапляє лише тоді, коли змінилися обидві сторони, і виходить звідти переходом resolve(keep) у «Synced». Видалення веде в кінцевий стан, а невдалий перенос залишає файл у попередньому стані з помилкою в SyncReport.errors.

5 Висновки

На етапі 1 побудовано UML-специфікацію системи Shelf: десять діаграм, що перевищує обов’язковий мінімум, фіксують дійових осіб, прецеденти, імена класів і модулів, правила статусів синхронізації та рішення обчислювати сортування, фільтр, вибір перегляду і план синхронізації на клієнті, у пакеті `@shelf/shared`. Це дає змогу покрити операції варіанта unit-тестами без сервера і використати ту саму логіку в десктопній та веб-версіях. Постачальників хостингу (Render, Neon, Supabase Storage, Vercel) обрано попередньо. Діаграми створено в draw.io, а звіт зібрано з Markdown конвертером pandoc у DOCX і PDF.